Bylaw 2023

Fall 2025

Motor Assignment

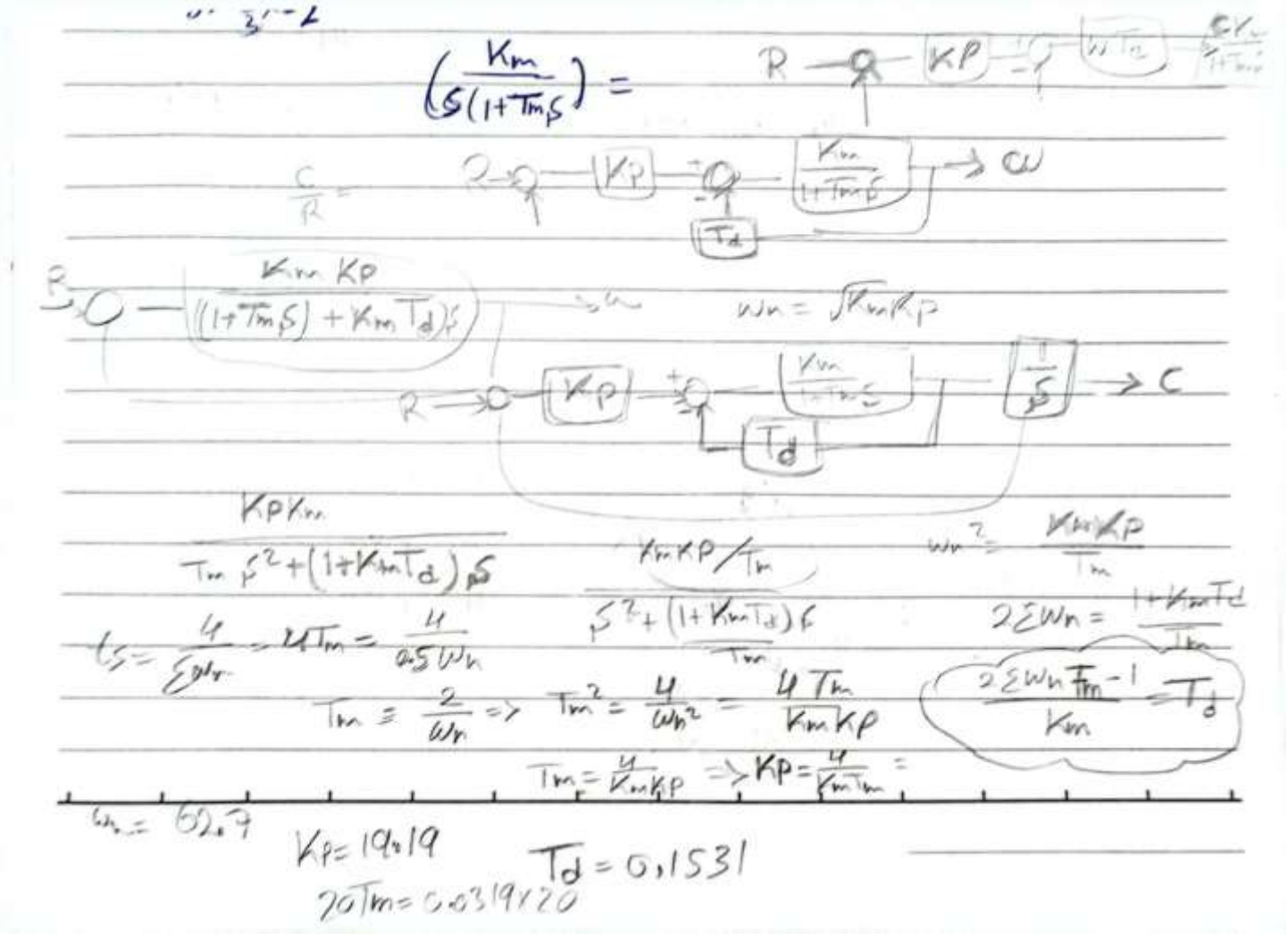
Phase 2



Name	ID
Abdelrahman Sherif Medhat	2300100

1. Calculate the required controller gains Determine the values of Kp and Td needed to achieve the desired system response based on the given damping ratio (ξ) and settling time (ts).

Hand analysis



2. .Verify and visualize the system response Simulate the system, then plot both:
   - o The output angle $c(t)$
   - o The controller output $u(t)$

Outputs

Design Requirements:

Target Damping (xi): 0.50

Target Settling Time: 0.1276 s

Calculated Controller Gains:

Kp = 19.1877

Td = 0.1531

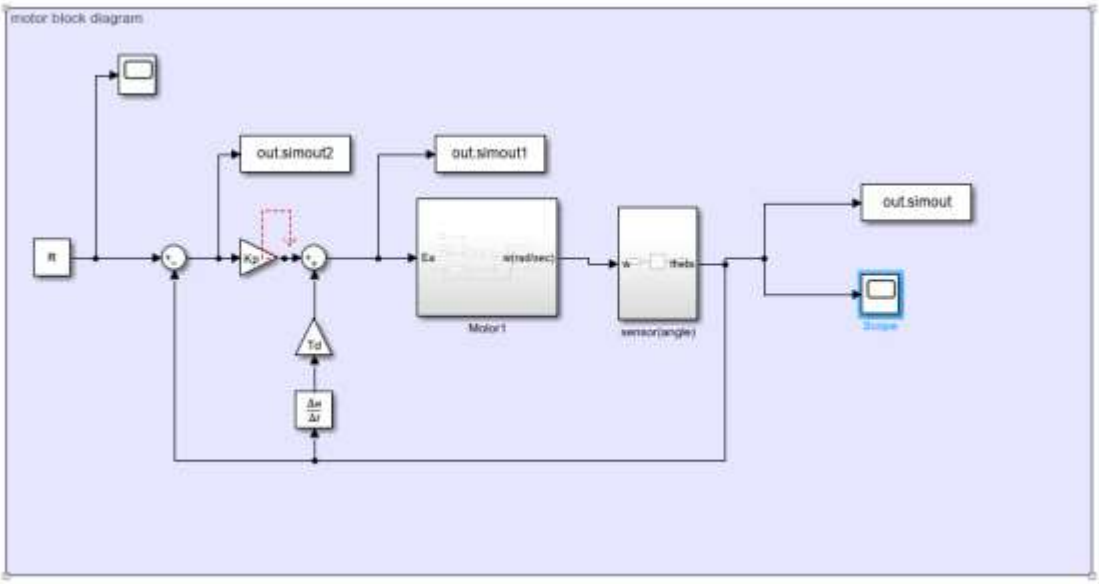
Verification:

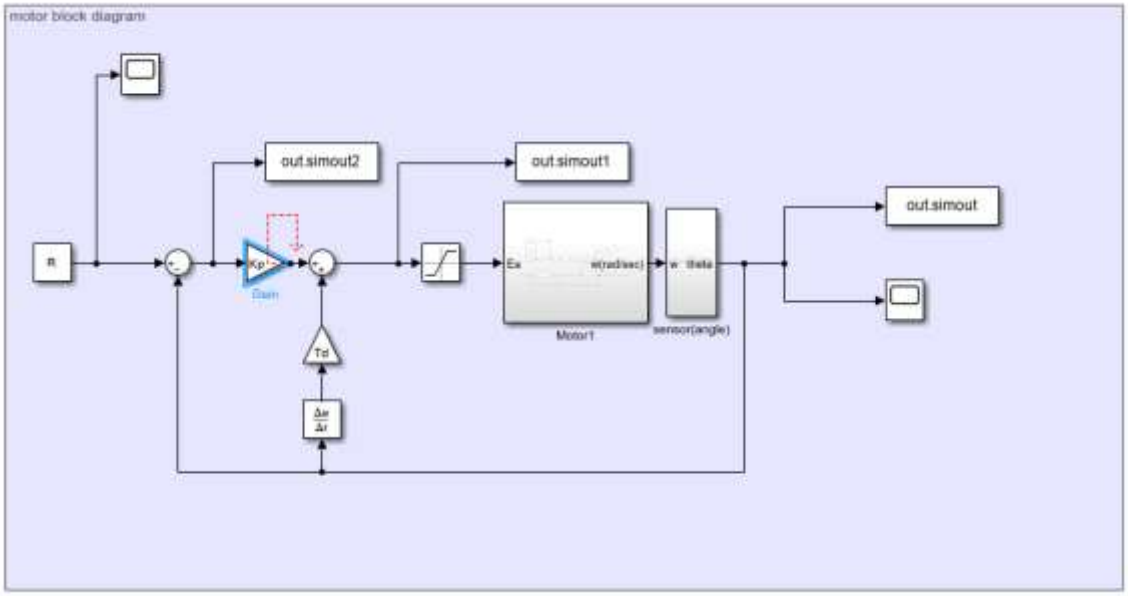
Resulting wn: 62.6819 rad/s

Resulting xi: 0.5000 (Target: 0.5)

Resulting ts: 0.1276 s (Target: 0.1276 s)

Block diagram





Matlab code

Editor - D:\reports\control systems\motor assignment\second_order_param.m *

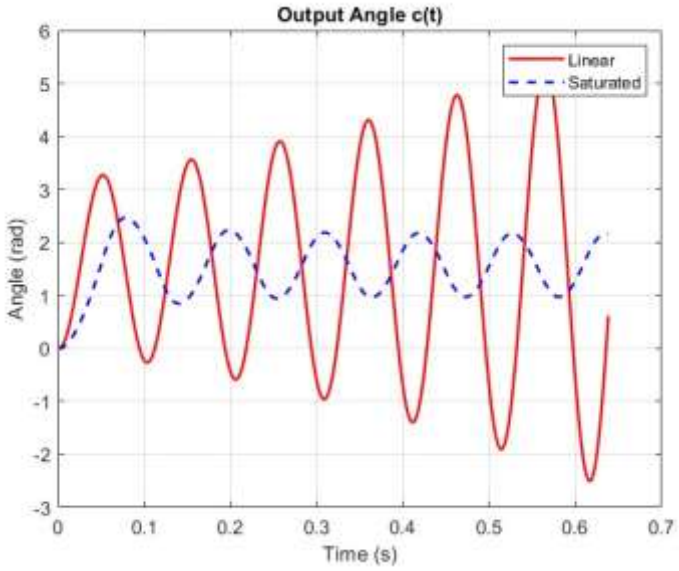
first_order_parm.m | motor_initialization.m | second_order_param.m *

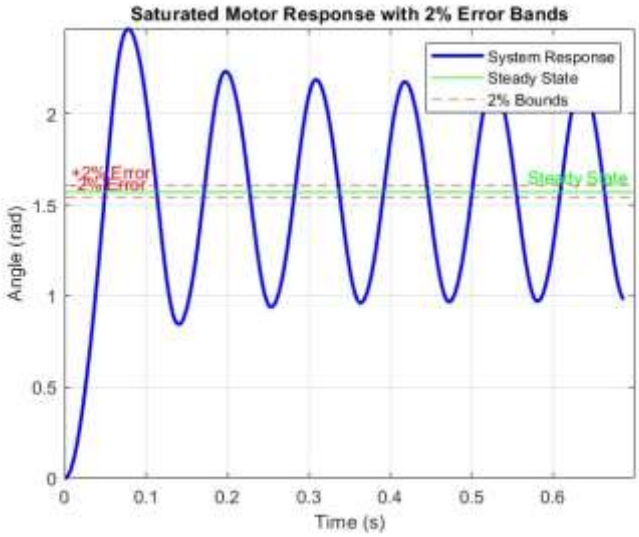
```
3   %  Constraints: Damping ratio (xi) = 0.5, Settling Time (ts) = 4*Tm
4   function [Kp,Td]=second_order_params(Km,Tm)
5   clc;
6   % 2. DESIGN REQUIREMENTS
7   xi = 0.5;  % Damping Ratio (zeta)
8   %%ts_target = 4 * Tm; % Target Settling Time (2% criterion)
9   ts_target= 4*Tm;
10  fprintf('----------------------------------------\n');
11  fprintf('Design Requirements:\n');
12  fprintf('   Target Damping (xi): %.2f\n', xi);
13  fprintf('   Target Settling Time: %.4f s\n', ts_target);
14  fprintf('----------------------------------------\n');
15  %CALCULATE GAINS
16  %Calculate required Natural Frequency (wn)
17  % Formula: ts = 4 / (xi * wn)  -->  wn = 4 / (xi * ts)
18  wn = 4 / (xi * ts_target);
19  %Calculate Td (Derivative Gain)
20  % Derivation: 2*xi*wn = (1 + K*Td) / Tm
21  % Rearranging for Td:
22  Td = (2 * xi * wn * Tm - 1) / Km;
23  %Calculate Kp (Proportional Gain)
24  % Derivation: wn^2 = (K * Kp) / Tm
25  % Rearranging for Kp:
26  Kp = (wn^2 * Tm) / Km;
27  % OUTPUT RESULTS
28  fprintf('Calculated Controller Gains:\n');
29  fprintf('   Kp = %.4f\n', Kp);
30  fprintf('   Td = %.4f\n', Td);
31  fprintf('----------------------------------------\n');
32
```

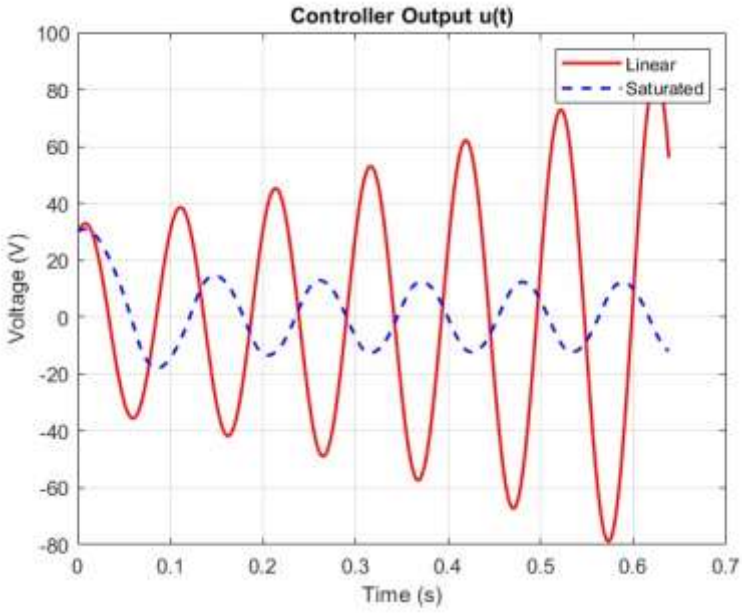
Editor - D:\reports\control systems\motor assignment\second_order_param.m

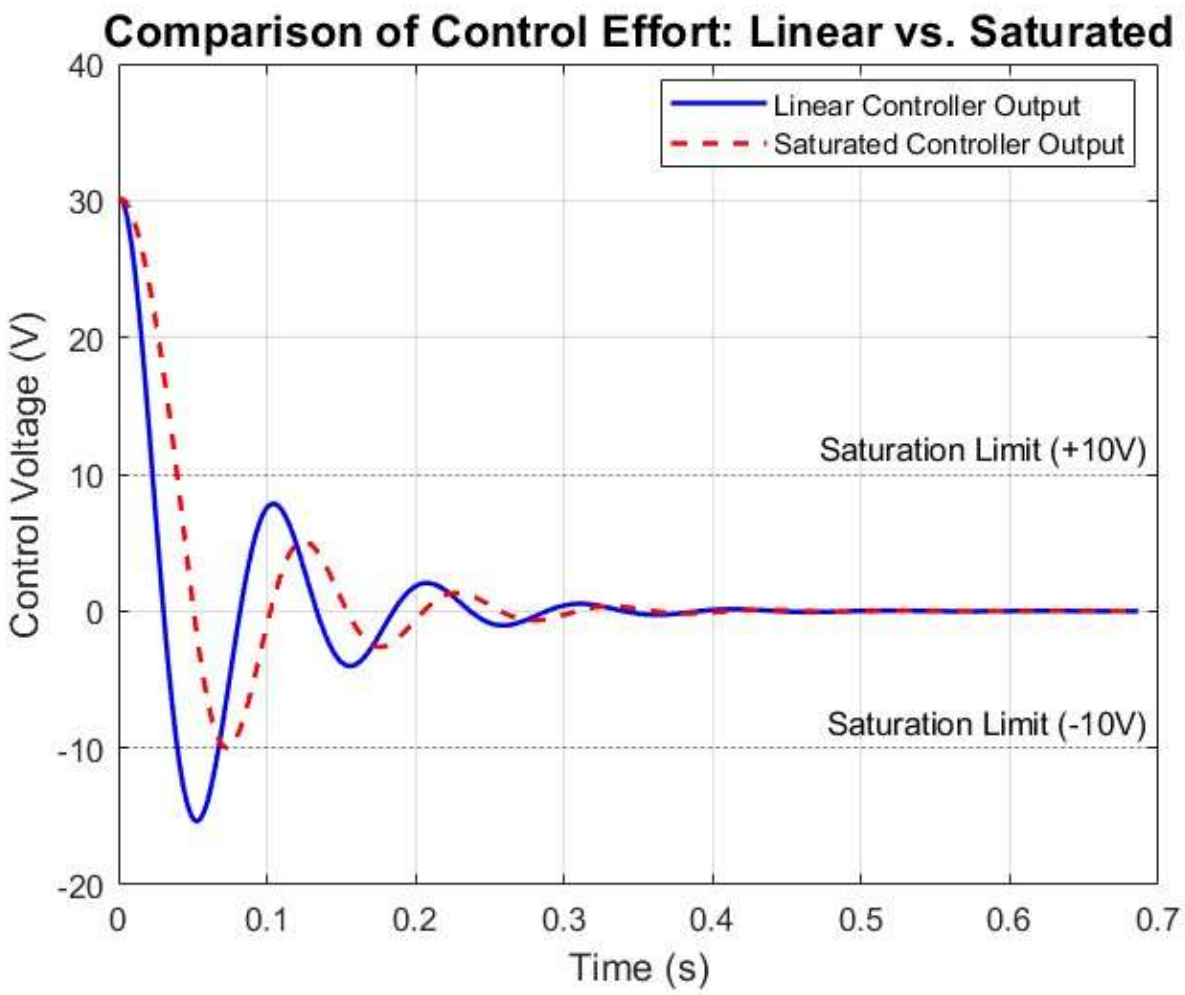
first_order_parm.m | motor_initialization.m | second_order_param.m

```
32
33  %VERIFICATION (Optional)
34
35  % This section verifies if the calculated gains theoretically meet the specs.
36  fprintf('Verification:\n');
37  actual_wn = sqrt((Km * Kp) / Tm);
38  actual_xi = (1 + Km * Td) / (2 * Tm * actual_wn);
39  actual_ts = 4 / (actual_xi * actual_wn);
40
41  fprintf('   Resulting wn: %.4f rad/s\n', actual_wn);
42  fprintf('   Resulting xi: %.4f (Target: 0.5)\n', actual_xi);
43  fprintf('   Resulting ts: %.4f s (Target: %.4f s)\n', actual_ts, ts_target);
44  end
45
46  [Kp,Td]=second_order_params(Km,Tm);
47
48
```


Output Angle c(t)
Linear
Saturated
Angle (rad)
Time (s)


Saturated Motor Response with 2% Error Bands
System Response
Steady State
2% Bounds
+2% Error
-2% Error
Steady State
Angle (rad)
Time (s)


Controller Output u(t)
Linear
Saturated
Voltage (V)
Time (s)


Comparison of Control Effort: Linear vs. Saturated
Linear Controller Output
Saturated Controller Output
Saturation Limit (+10V)
Saturation Limit (-10V)
Control Voltage (V)
Time (s)
40
30
20
10
0
-10
-20
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7

3. Develop a MATLAB script to compute the IAE Write a script that calculates the Integral of Absolute Error (IAE) as defined below, and compare the resulting IAE values with and without controller saturation:

Editor - D:\reports\control systems\motor assignment\comparing_motors_outputs.m *

first_order_parm.m | motor_initialization.m | second_order_param.m | iae.m | optimize.m

```
R=pi/2;
sim_time=20*Tm;
function compare_motors
    clc;close all;


    global Ka Kb Ra La b Ea Tf
    Ka = 1e-2;
    Kb = 1e-1;
    Ra = 1e-1;
    La = 1e-2;
    b  = 5e-3;
    Ea = 10;   % Note: In closed loop, this might be overridden by the contr
    Tf = 2.0;  % Simulation time (increased to 2s to see settling)


    res_lin = sim("figure3_motor.slx");

    t_lin     = res_lin.tout;       % Time vector
    theta_lin = res_lin.simout;     % Output Angle (c)
    u_lin     = res_lin.simout1;    % Controller Output (u)
    e_lin     = res_lin.simout2;    % Error Signal (e)

    res_sat = sim("figure4_motor.slx");

    t_sat     = res_sat.tout;       % Time vector
    theta_sat = res_sat.simout;     % Output Angle (c)
    u_sat     = res_sat.simout1;    % Controller Output (u)
    e_sat     = res_sat.simout2;    % Error Signal (e)


    figure('Name', 'Comparison: Angle');

```

Editor - D:\reports\control systems\motor assignment\comparing_motors_outputs.m

first_order_parm.m | motor_initialization.m | second_order_param.m | iae.m | optimize.m | figure7.m

```
34
35      if isa(theta_lin, 'timeseries'), y1 = theta_lin.Data; else, y1 = theta_lin; end
36      if isa(theta_sat, 'timeseries'), y2 = theta_sat.Data; else, y2 = theta_sat; end
37
38      plot(t_lin, y1, 'r', 'LineWidth', 1.5); hold on;
39      plot(t_sat, y2, 'b--', 'LineWidth', 1.5);
40      grid on;
41      legend('Linear', 'Saturated');
42      title('Output Angle c(t)');
43      xlabel('Time (s)'); ylabel('Angle (rad)');
44
45      figure('Name', 'Comparison: Control Signal');
46
47      if isa(u_lin, 'timeseries'), u1 = u_lin.Data; else, u1 = u_lin; end
48      if isa(u_sat, 'timeseries'), u2 = u_sat.Data; else, u2 = u_sat; end
49
50      plot(t_lin, u1, 'r', 'LineWidth', 1.5); hold on;
51      plot(t_sat, u2, 'b--', 'LineWidth', 1.5);
52      grid on;
53      legend('Linear', 'Saturated');
54      title('Controller Output u(t)');
55      xlabel('Time (s)'); ylabel('Voltage (V)');
56
57
58
59      function score = calculate_IAE(time_vec, error_obj)
60
61          if isa(error_obj, 'timeseries')
62              e_data = error_obj.Data;
63          else
64              e_data = error_obj;
65          end
```

Editor - D:\reports\control systems\motor assignment\comparing_motors_outputs.m

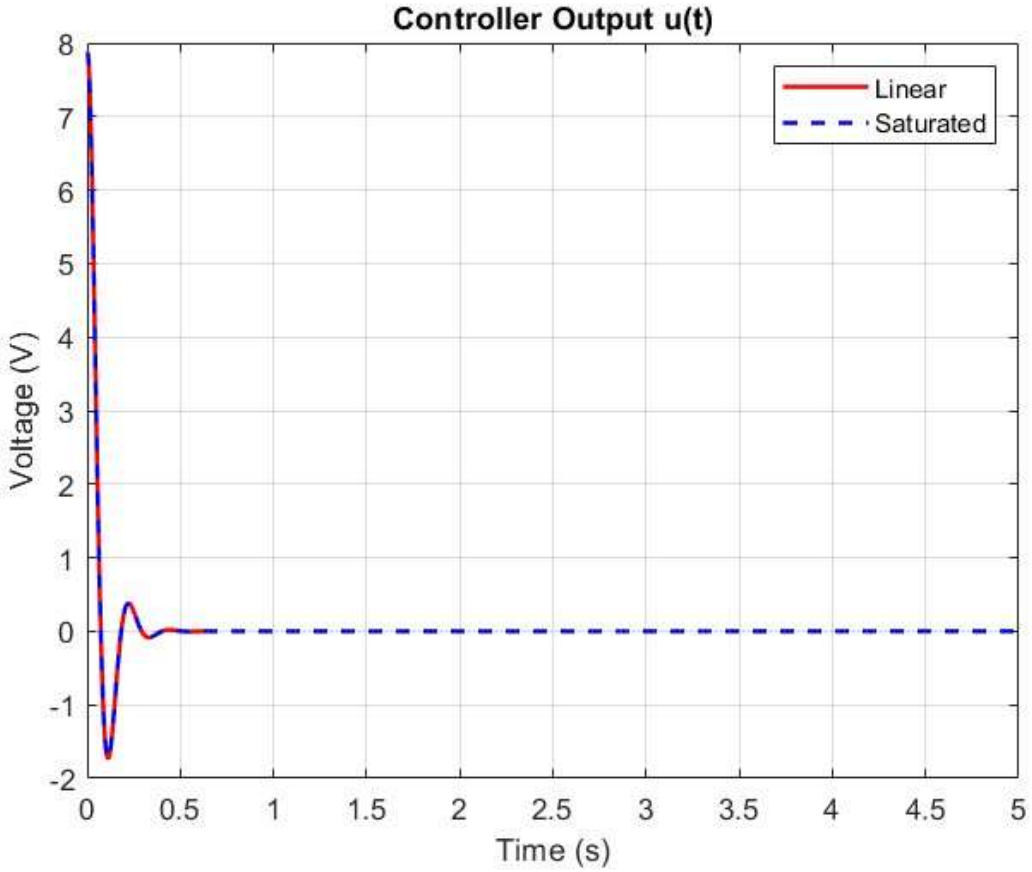
first_order_parm.m | motor_initialization.m | second_order_param.m | iae.m | optimize.m

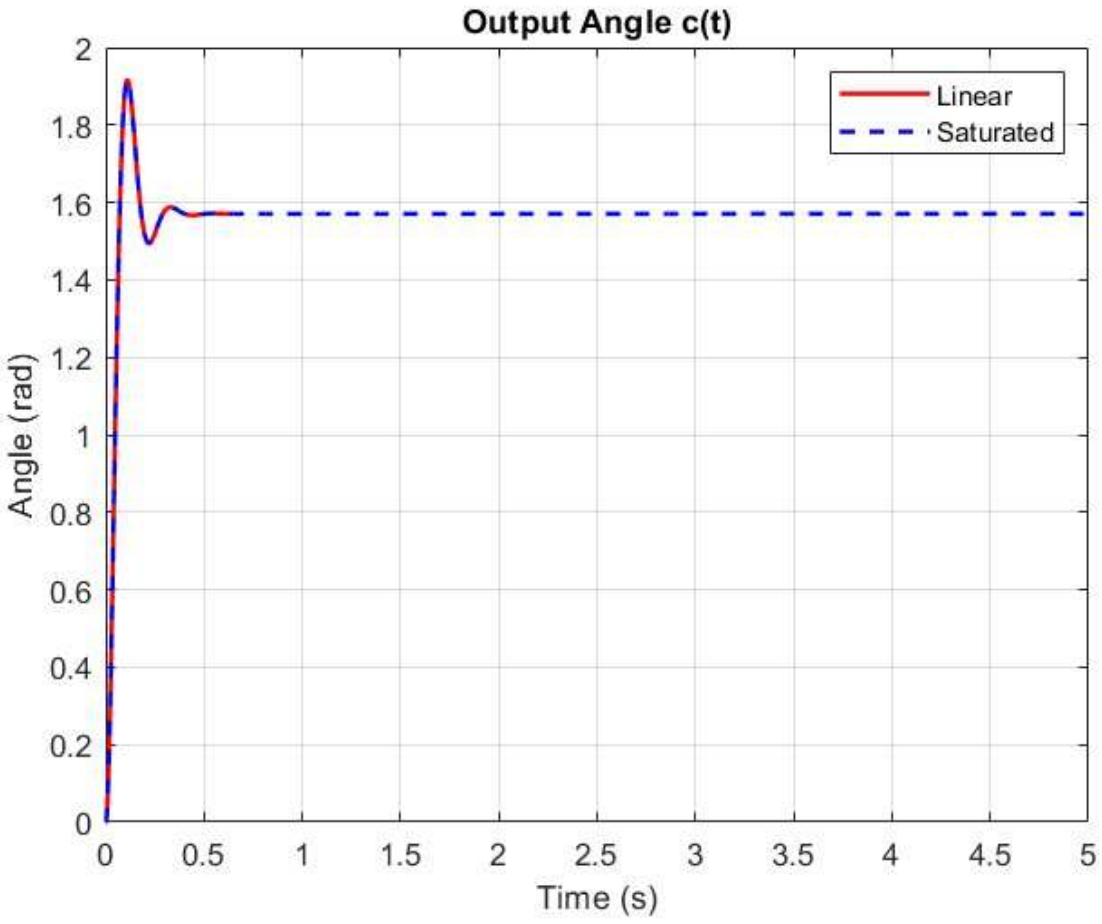
```
56
57
58
59      function score = calculate_IAE(time_vec, error_obj)
60
61          if isa(error_obj, 'timeseries')
62              e_data = error_obj.Data;
63          else
64              e_data = error_obj;
65          end
66
67
68          dt = diff(time_vec);
69
70
71          score = sum(abs(e_data(1:end-1)) .* dt);
72      end
73
74
75      iae_linear = calculate_IAE(t_lin, e_lin);
76      iae_sat    = calculate_IAE(t_sat, e_sat);
77
78      fprintf('-------------------------------------------------\n');
79      fprintf('PART F RESULTS: IAE Comparison\n');
80      fprintf('-------------------------------------------------\n');
81      fprintf('Linear Model IAE    : %.4f\n', iae_linear);
82      fprintf('Saturated Model IAE : %.4f\n', iae_sat);
83      fprintf('-------------------------------------------------\n');
84  end
85  compare_motors;
86
```

PART F RESULTS: IAE Comparison

Linear Model IAE   : 0.0936

Saturated Model IAE : 0.0937


Controller Output u(t)
Linear
Saturated
Voltage (V)
Time (s)
8
7
6
5
4
3
2
1
0
-1
-2
0
0.5
1
1.5
2
2.5
3
3.5
4
4.5
5


Output Angle c(t)
Linear
Saturated
Angle (rad)
Time (s)
2
1.8
1.6
1.4
1.2
1
0.8
0.6
0.4
0.2
0
0
0.5
1
1.5
2
2.5
3
3.5
4
4.5
5